Decision Trees and the Bootstrap

Week 7

Learning Outcomes

By the end of this week, you should be able to:

1. Explain how decision trees partition the feature space to make predictions for regression and classification.
2. Describe how recursive binary splitting constructs decision trees and how splitting criteria differ between regression and classification.
3. Explain how cost-complexity pruning and cross-validation help prevent overfitting.
4. Evaluate the advantages and limitations of decision trees compared to linear models.
5. Describe the bootstrap procedure and how it can be used to estimate the uncertainty of a statistical estimator.

Tree-Based Methods

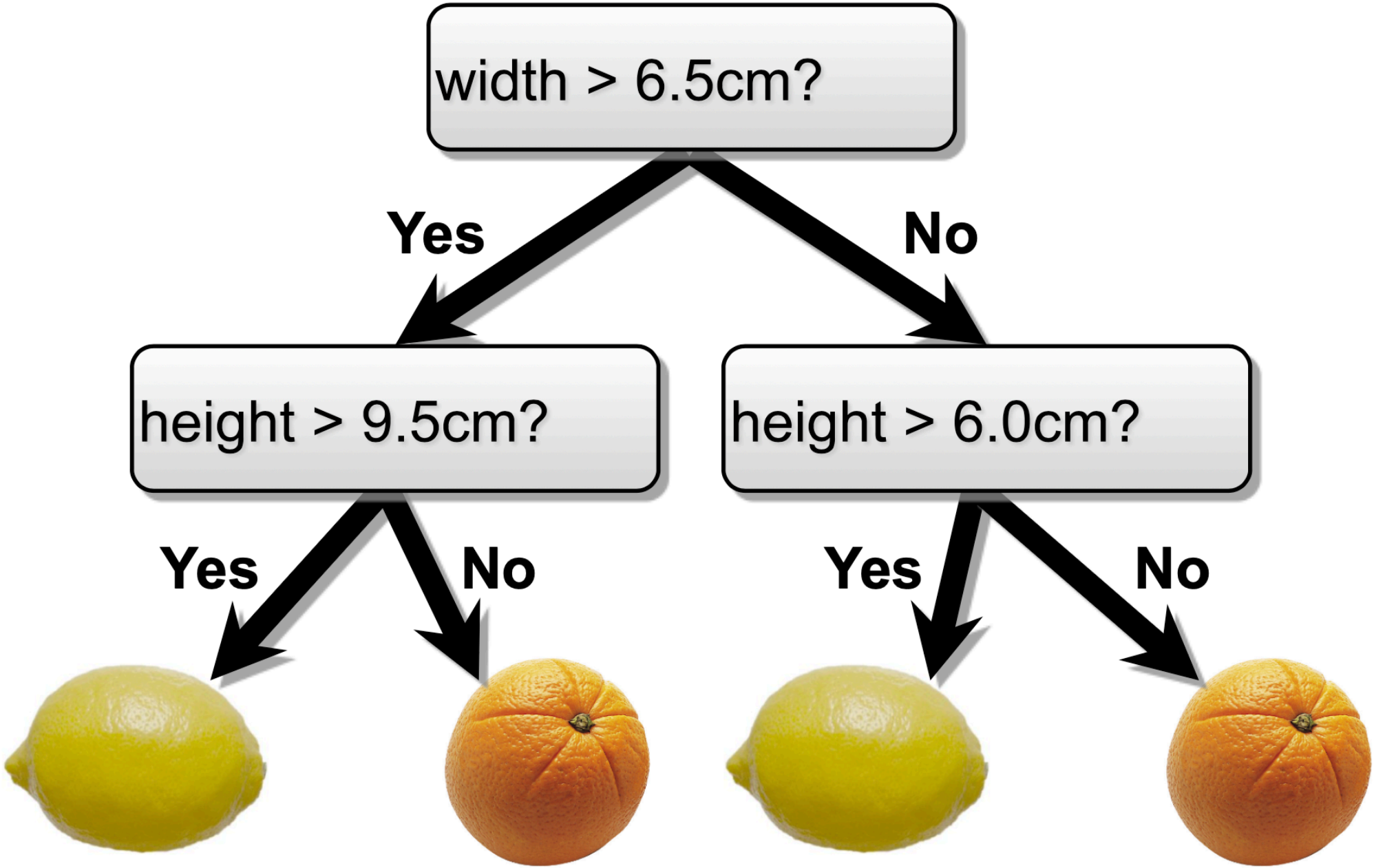
Tree-Based Methods

- Tree-based methods can be applied for both regression and classification.
- Since the method can be summarized in a tree structure, these types of approaches are known as **decision tree methods**.

- Tree-based methods are simple and useful for interpretation.
- However, they typically are not competitive with the best supervised learning approaches.
- We will (later) introduce bagging, random forests, and boosting to combine multiple decision trees to improve the performance.

Example of Decision Trees

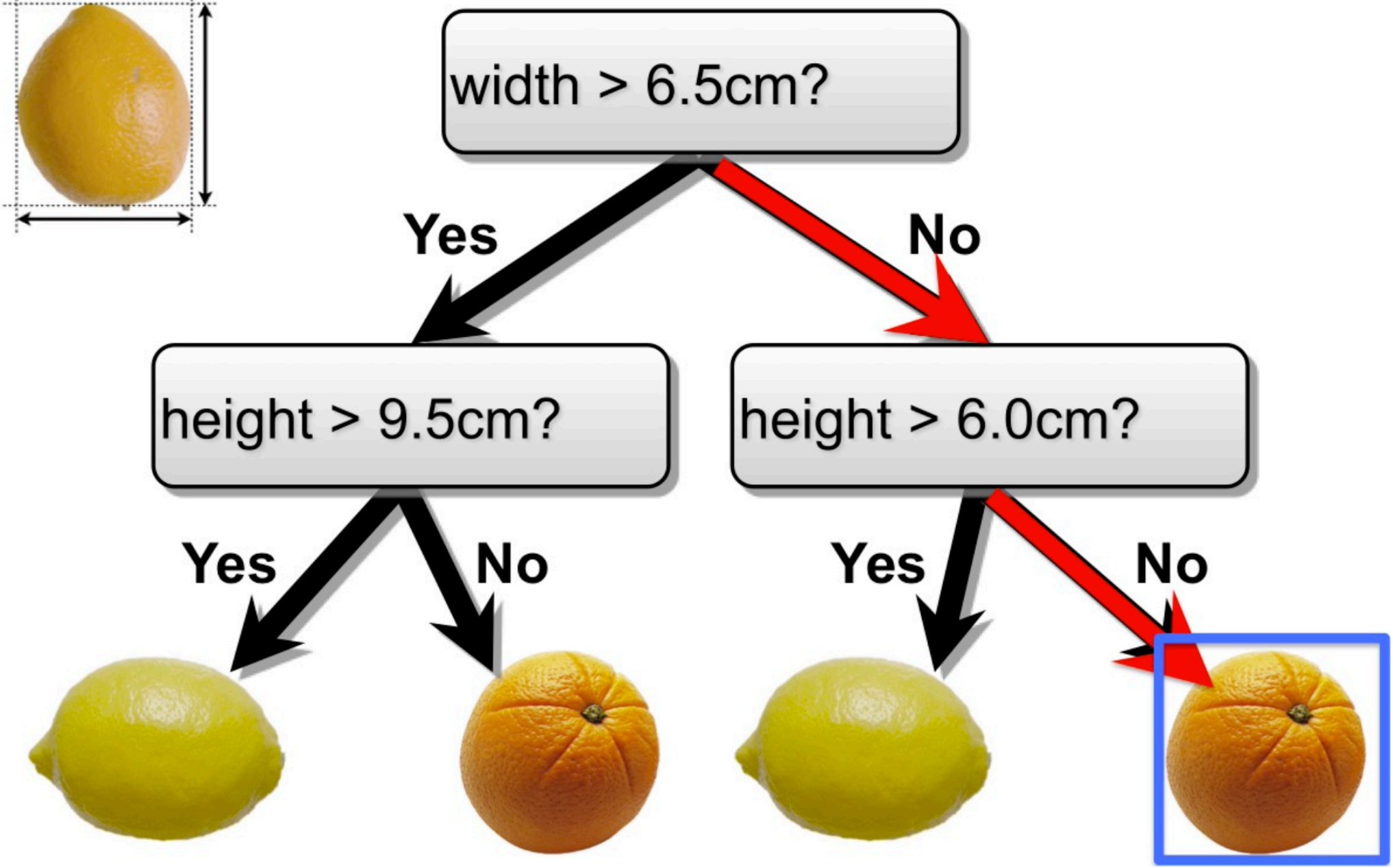
Make predictions by splitting on features according to a tree structure.


width > 6.5cm?
Yes
No
height > 9.5cm?
height > 6.0cm?
Yes
No
Yes
No

Example of Decision Trees

Make predictions by splitting on features according to a tree structure.

Test example



Example of Decision Trees—Discrete features

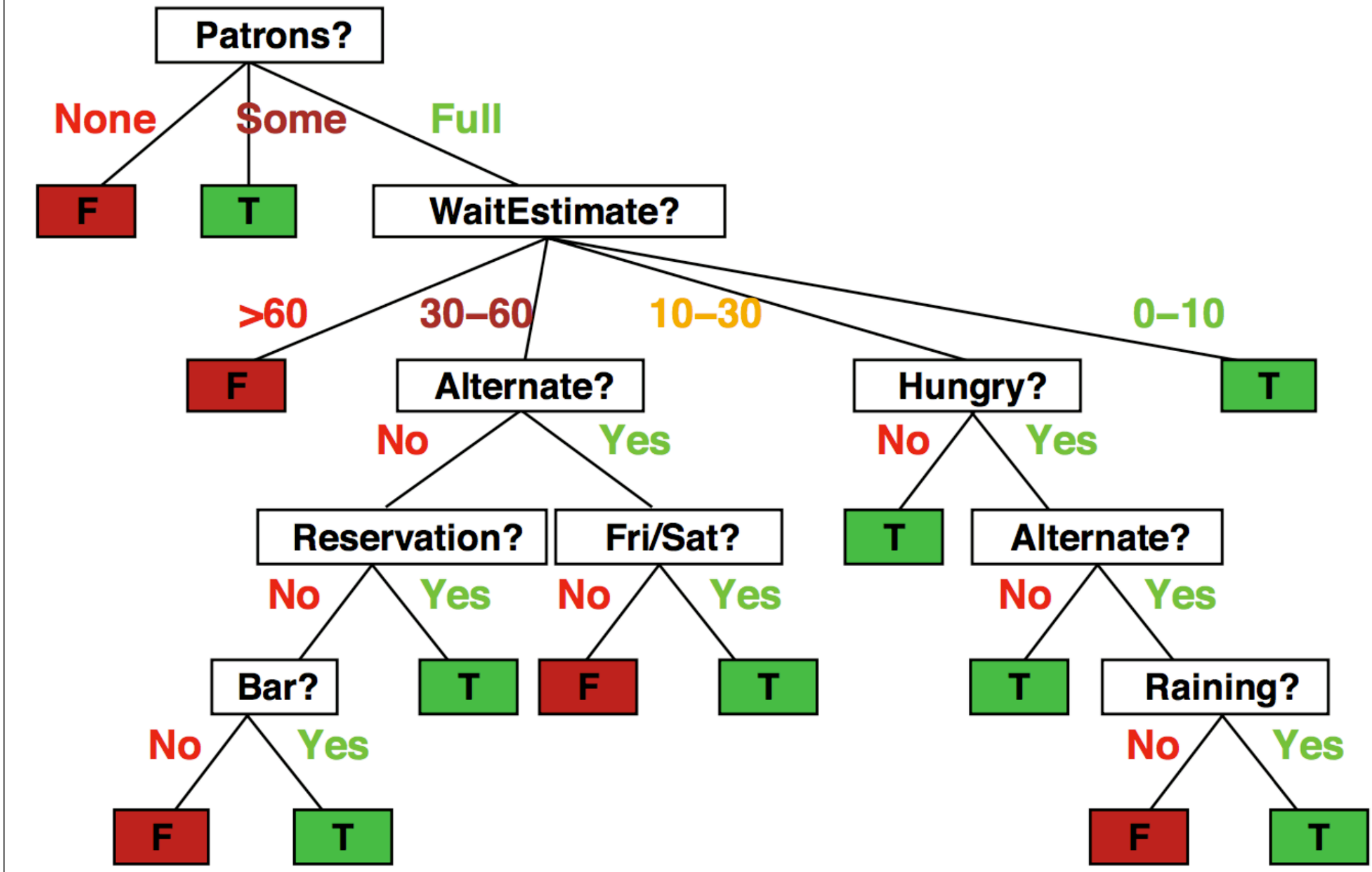
Example	Input Attributes										Goal
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

Example of Decision Trees—Discrete features

1.	Alternate: whether there is a suitable alternative restaurant nearby.
2.	Bar: whether the restaurant has a comfortable bar area to wait in.
3.	Fri/Sat: true on Fridays and Saturdays.
4.	Hungry: whether we are hungry.
5.	Patrons: how many people are in the restaurant (values are None, Some, and Full).
6.	Price: the restaurant's price range ($, $$, $$$).
7.	Raining: whether it is raining outside.
8.	Reservation: whether we made a reservation.
9.	Type: the kind of restaurant (French, Italian, Thai or Burger).
10.	WaitEstimate: the wait estimated by the host (0-10 minutes, 10-30, 30-60, >60).

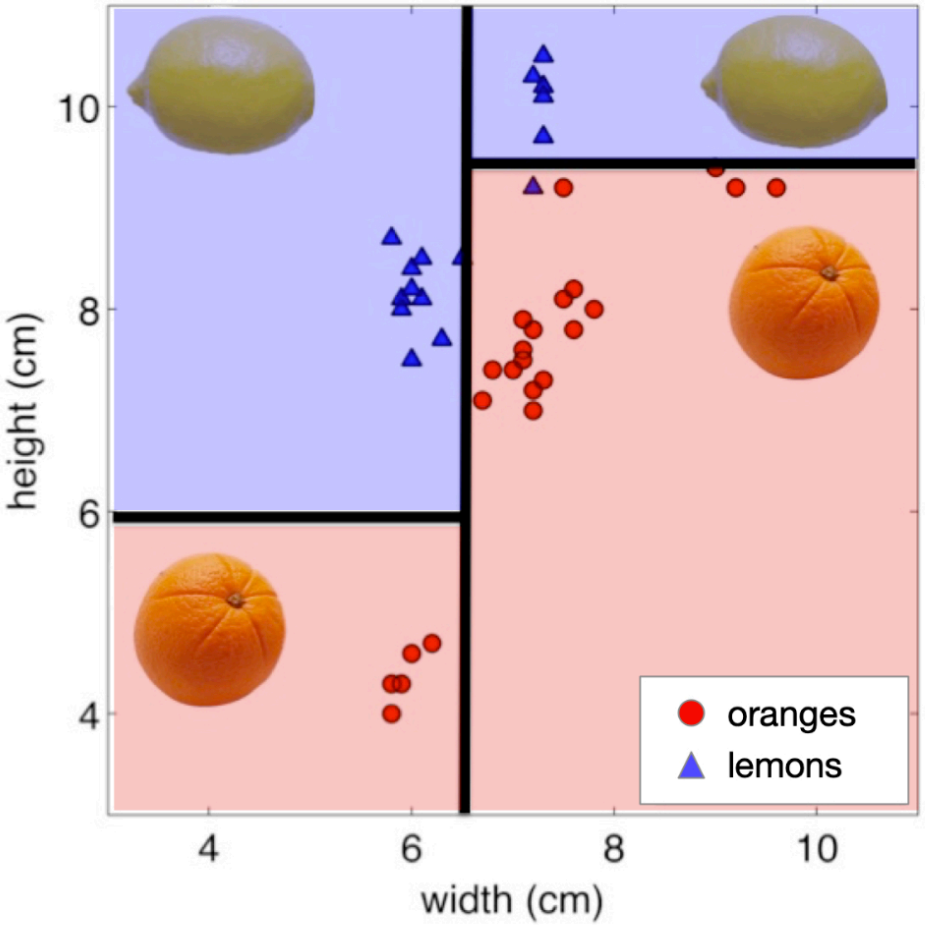
Example of Decision Trees—Discrete features

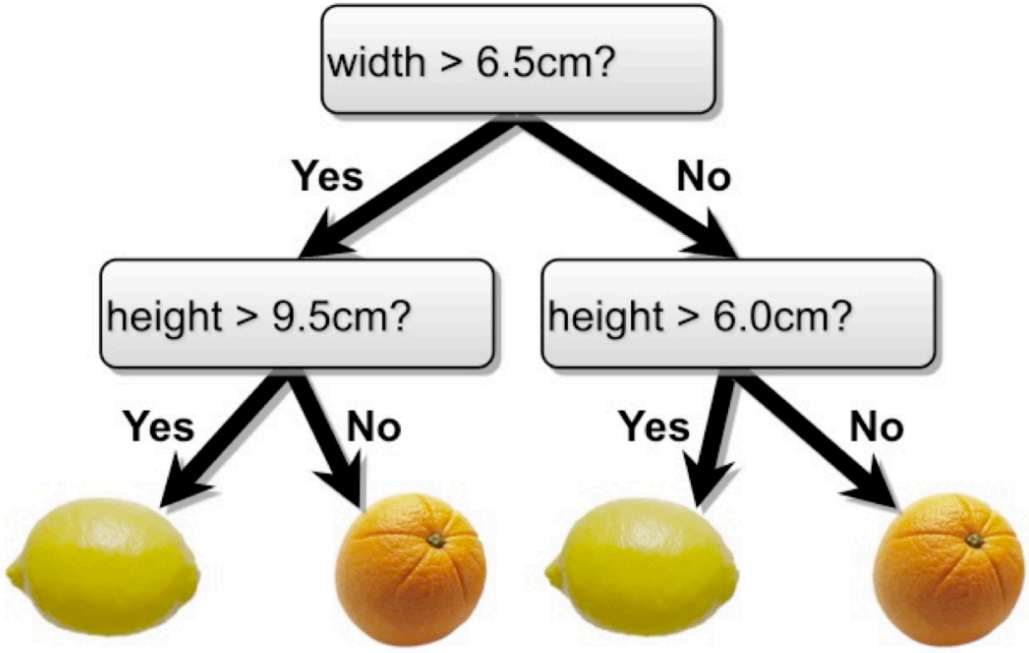
Split discrete features into a partition of possible values.


Patrons?
None
Some
Full
F
T
WaitEstimate?
>60
30–60
10–30
0–10
F
Alternate?
Hungry?
T
No
Yes
No
Yes
Reservation?
Fri/Sat?
T
Alternate?
No
Yes
No
Yes
No
Yes
Bar?
T
F
T
T
Raining?
No
Yes
No
Yes
F
T
F
T

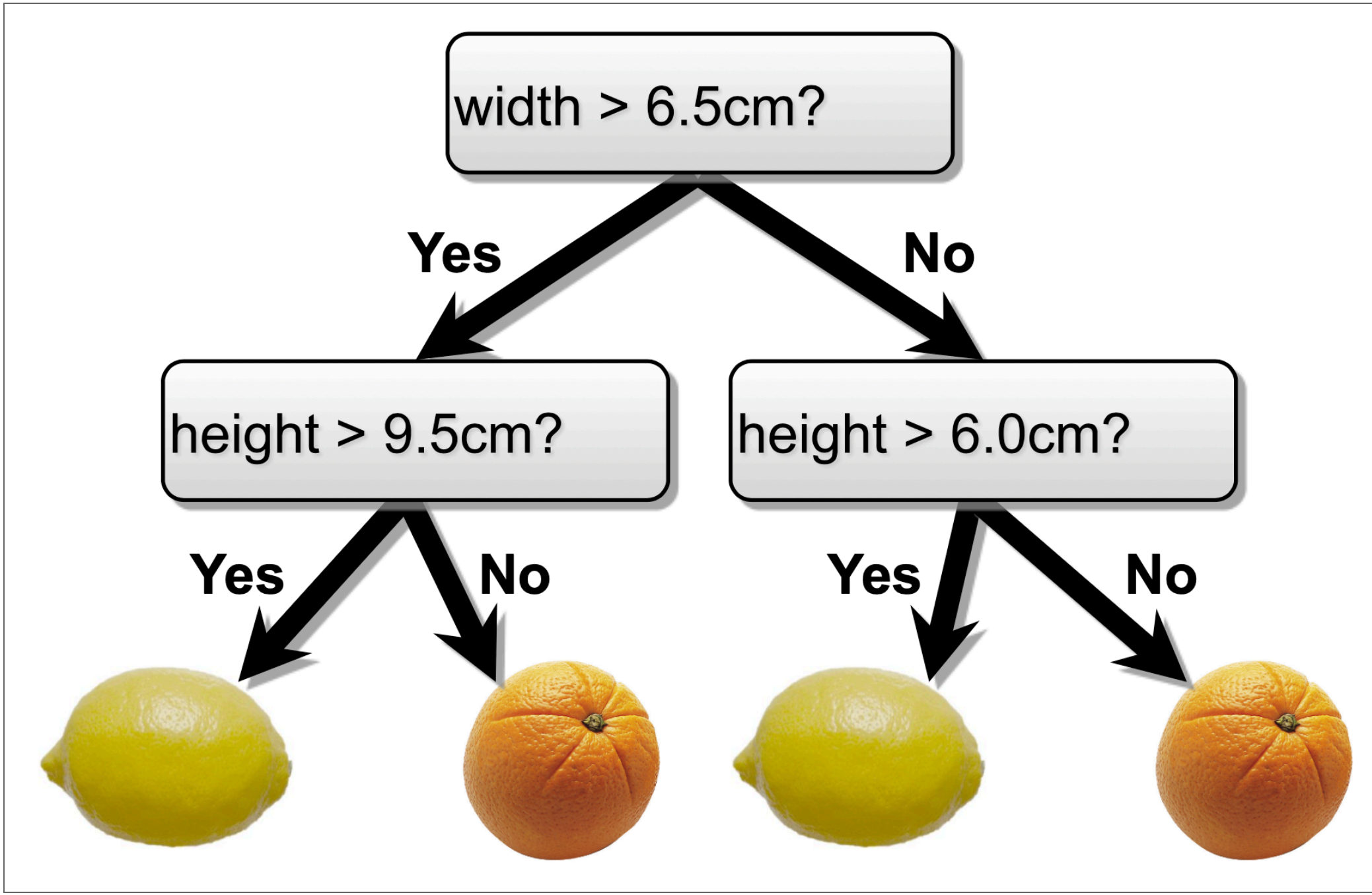
Example of Decision Trees—Continuous features

For continuous features, we partition the range by checking whether that attribute is greater than or less than some threshold.
Decision boundary is made up of rectangles.


10
8
6
4
height (cm)
4
6
8
10
width (cm)
oranges
lemons


width > 6.5cm?
Yes
No
height > 9.5cm?
height > 6.0cm?
Yes
No
Yes
No

Terminology of Decision Trees



- An internal node means an attribute, i.e., a feature.
- Branching is determined by the attribute value.
- Children of a node partition the range of the attribute from the parent.
- Leaf nodes are predicted outputs.

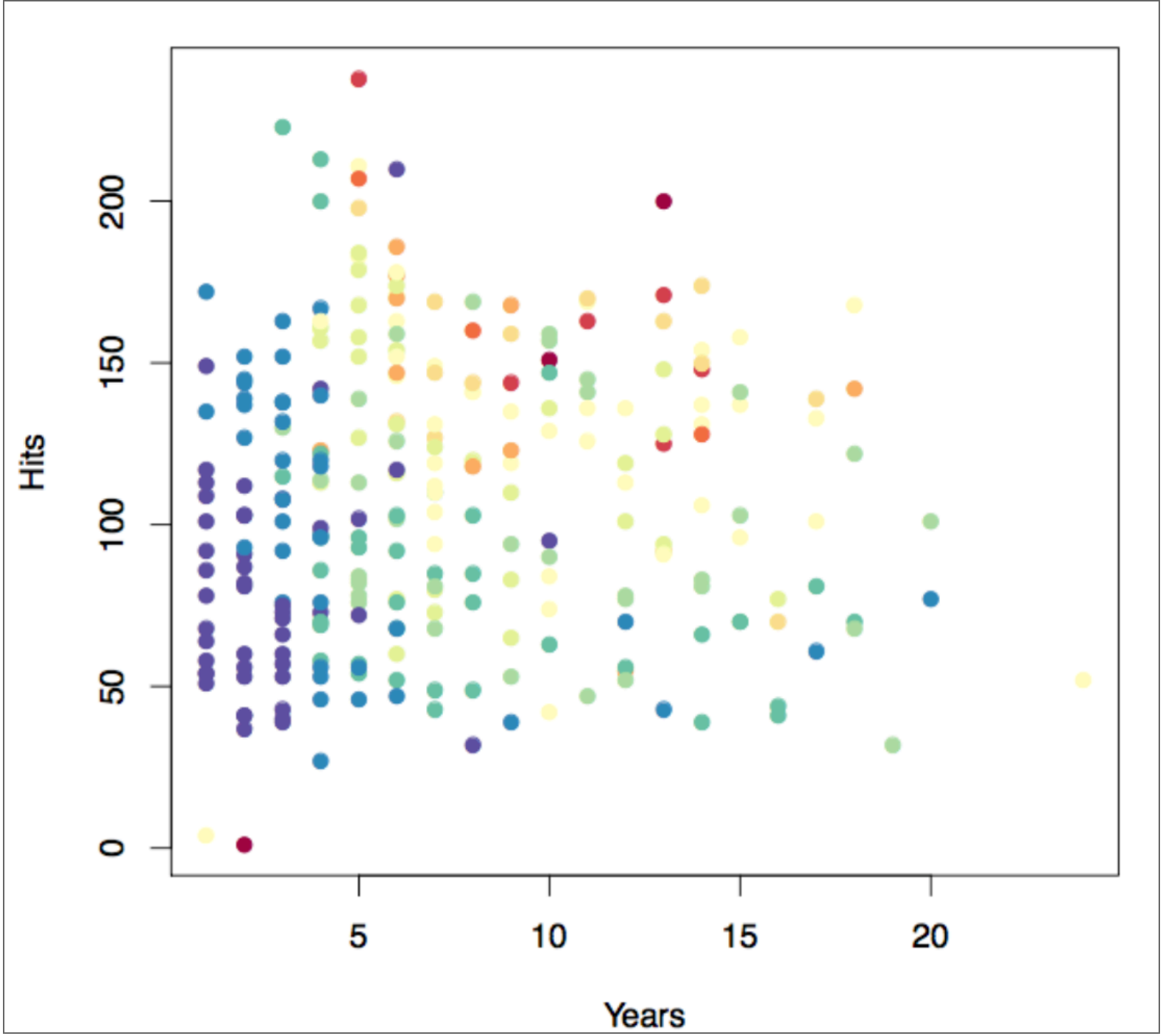
Decision Trees—Classification and Regression

- Let J be the total number of paths from root to a leaf.
- Each path, $j \in \{1, 2, \ldots, J\}$, defines a region R_j of the feature space.
- Let $\{(x_1^{(j)}, y_1^{(j)}), \ldots, (x_k^{(j)}, y_k^{(j)})\}$ be the training data points that fall into R_j

- **Regression tree:**
  - continuous output
  - leaf value y^{R_j} typically set to the mean value in $\{y_1^{(j)}, \ldots, y_k^{(j)}\}$
- **Classification tree:**
  - discrete output
  - leaf value y^{R_j} typically set to the majority label in $\{y_1^{(j)}, \ldots, y_k^{(j)}\}$

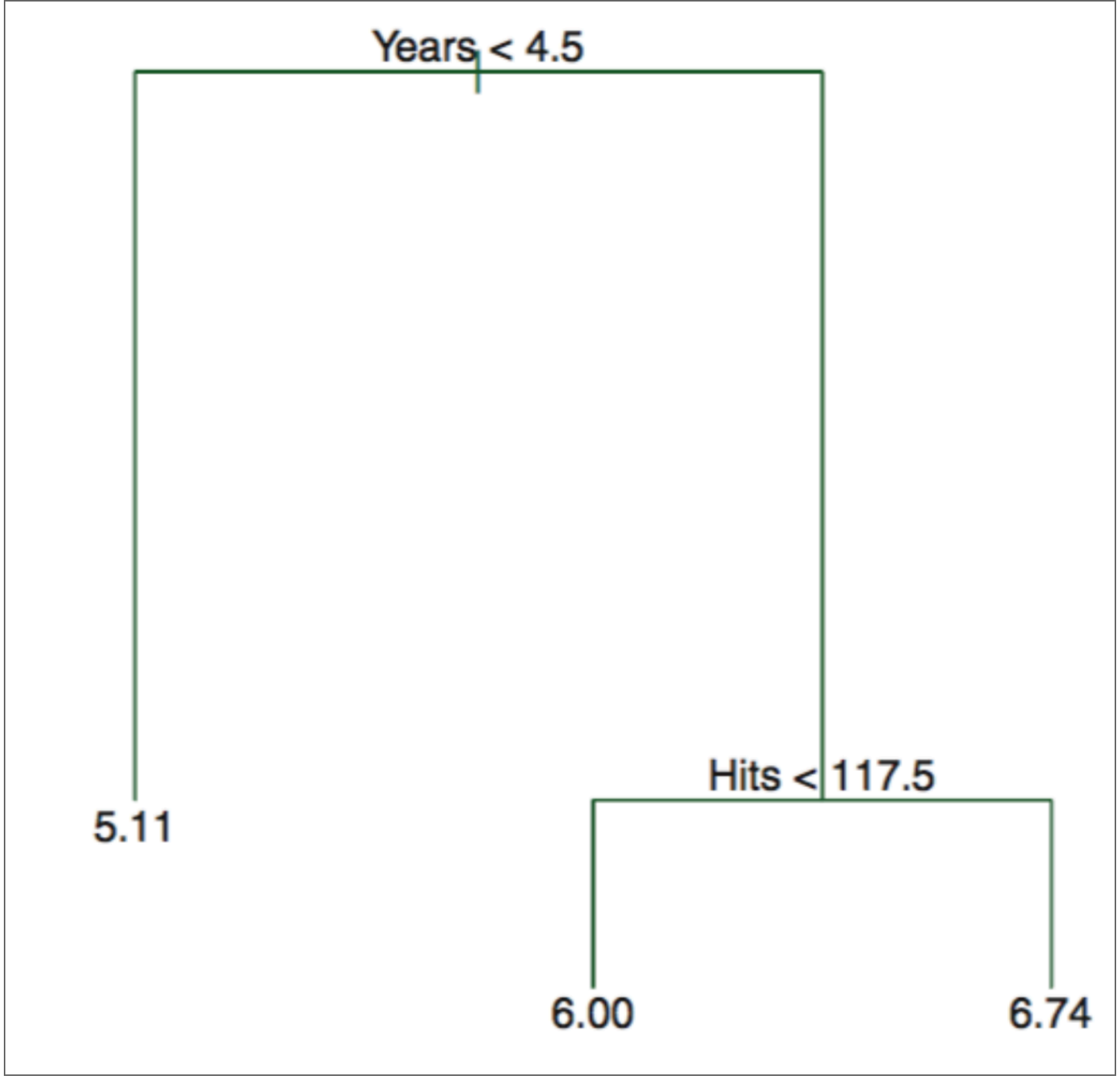
Regression Decision Trees

We want to predict the salary of a baseball player, based on (1) the number of years that he has played in the leagues and (2) the number of hits that he made in the previous year.



Salary is color-coded from low (blue, green) to high (yellow, red).

What does a fitted decision tree look like?



At a given internal node, the label (of the form $X_j < t$) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to $X_j \geq t$. The

number in each leaf (external node) is the mean of the response for the observations that fall there.

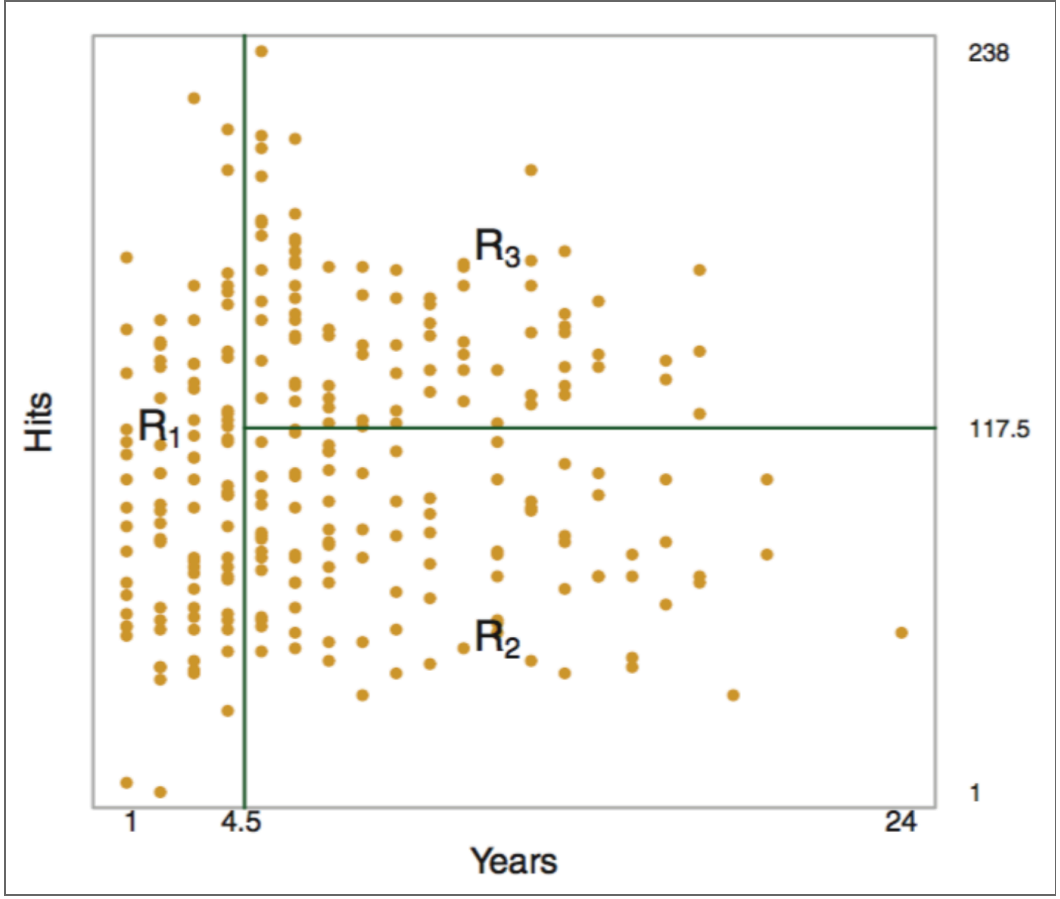
Partitioned regions for the Hitters data set

Overall, the tree stratifies or segments the players into three regions of feature space:

$$R_1 = \{X \mid \text{Years} < 4.5\}$$

$$R_2 = \{X \mid \text{Years} \geq 4.5,\ \text{Hits} < 117.5\}$$

$$R_3 = \{X \mid \text{Years} \geq 4.5,\ \text{Hits} \geq 117.5\}.$$


238
117.5
1
Hits
R_1
R_2
R_3
1
4.5
24
Years

Interpretation of Results

- Years is the most important factor in determining Salary, and players with less experience earn lower salaries than more experienced players.

- Given that a player is less experienced, the number of Hits that he made in the previous year seems to play little role in his Salary.

- But among players who have been in the major leagues for five or more years, the number of Hits made in the previous year does affect Salary, and players who made more Hits last year tend to have higher salaries.

- Surely an over-simplification, but compared to a regression model, it is easier to interpret, and has a nice graphical representation.

How to Build a Regression Tree?

- **Step 1:** We divide the feature space, i.e., the set of possible values for $X_1, X_2, \ldots, X_p$, into J distinct and non-overlapping regions, $R_1, R_2, \ldots, R_J$.

- **Step 2:** For every observation that falls into the region R_j, we make the same prediction, which is simply the mean of the response values for the training observations in R_j.

How to Construct Regions $R_1, R_2, \ldots, R_J$?

- In theory, the splitted regions could have any shape. However, we choose to divide the feature space into multi-dimensional rectangles for ease of interpretation of the resulting predictive model.
- The goal is to find rectangles $R_1, R_2, \ldots, R_J$ that minimize the residual sum of squares (RSS), given by

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2,$$

where

$$\bar{y}_{R_j} = \frac{1}{|R_j|} \sum_{i \in R_j} y_i$$

is the mean response for the training observations in R_j.

How to Construct Regions $R_1, R_2, \ldots, R_J$?

- Unfortunately, it is computationally infeasible to enumerate all possible partitions of the feature space into J boxes.

- For this reason, we take a top-down, greedy approach that is known as **recursive binary splitting**.

- The approach is top-down because it begins at the top of the tree and then successively splits the feature space; each split is indicated via two new branches further down on the tree.

- It is greedy because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

How to Construct Regions $R_1, R_2, \ldots, R_J$?

- We first select the feature X_j among all $X_1, \ldots, X_p$, and the cutpoint s such that splitting the feature space into the regions $\{X \mid X_j < s\}$ and $\{X \mid X_j \geq s\}$ leads to the greatest possible reduction in RSS.

- Mathematically, we seek the value of $j \in \{1, 2, \ldots, p\}$ and $s \in \mathbb{R}$ that minimize the equation

$$\sum_{i:x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2$$

where

- $R_1(j, s) = \{X \mid X_j < s\}$ and $R_2(j, s) = \{X \mid X_j \geq s\}$
- $\bar{y}_{R_1}$ and $\bar{y}_{R_2}$ are, respectively, the averaged responses of the training data in $R_1(j, s)$ and $R_2(j, s)$.

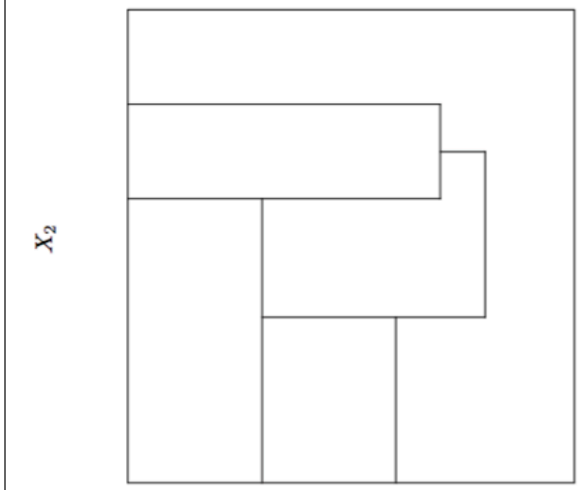
How to Construct Regions $R_1, R_2, \ldots, R_J$?

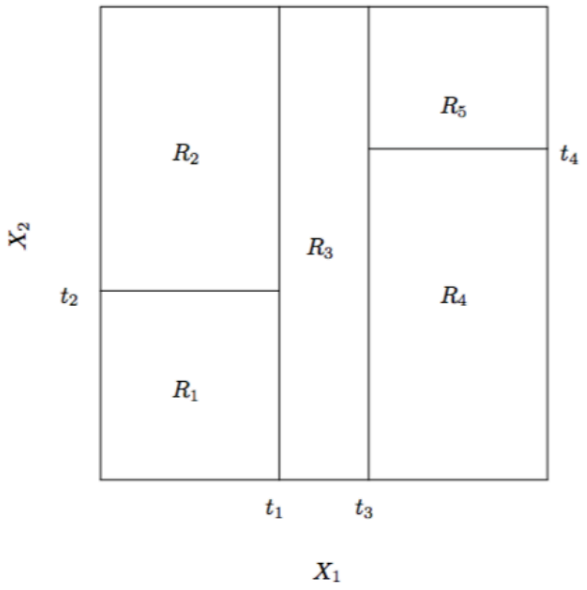
- Next, we repeat the process, looking for the best feature and the best cutpoint in order to further split the data, leading to the greatest reduction of RSS.
- However, this time, instead of splitting the entire feature space, we split one of the two previously identified regions. We now have three regions.

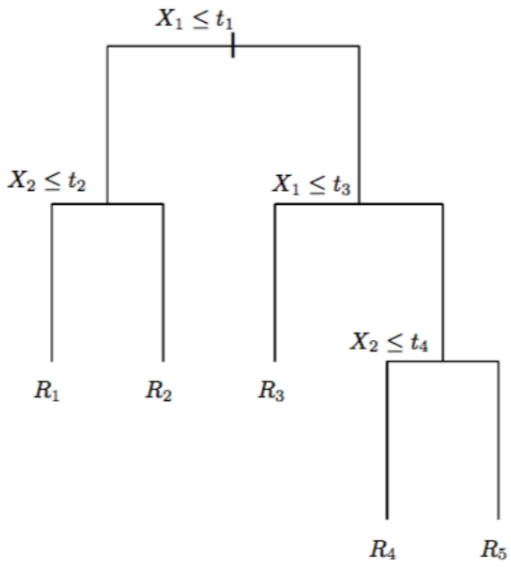
- The process continues until a stopping criterion is reached
- we may stop when no region contains more than 5 observations

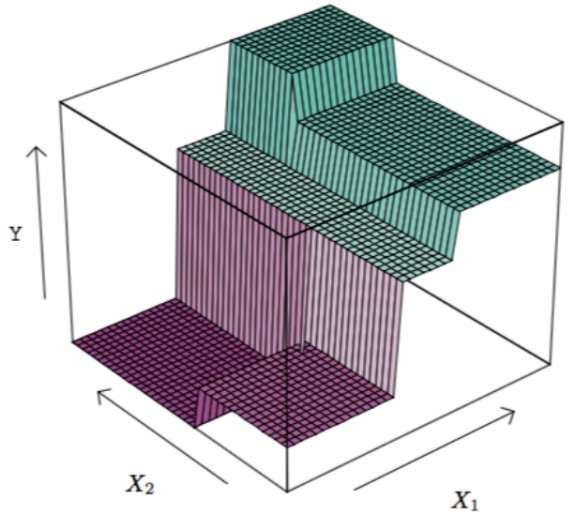
- Once $R_1, R_2, \ldots, R_J$ are identified, we predict the response for a given test point using the averaged response of the training observations in the region.

An Example with 5 Regions


X_2
X_1


R_1
R_2
R_3
R_4
R_5
t_1
t_2
t_3
t_4
X_2
X_1


$X_1 \leq t_1$
$X_2 \leq t_2$
$X_1 \leq t_3$
$X_2 \leq t_4$
R_1
R_2
R_3
R_4
R_5


Y
X_2
X_1

An Example with 5 Regions

- **Top Left**: A partition of two-dimensional feature space that could not result from recursive binary splitting.
- **Top Right**: The output of recursive binary splitting on a two-dimensional example.
- **Bottom Left**: A tree corresponding to the partition in the top right panel.
- **Bottom Right**: A perspective plot of the prediction surface corresponding to that tree.

Tree Pruning

- The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance.
- This can happen when the resulting decision tree is too complex (e.g. only one observation per region)

- A smaller tree with fewer splits (that is, fewer regions $R_1, R_2, \ldots, R_J$) might lead to lower variance and better interpretation at the cost of a little bias.

- Intuitively, our goal is to select a subtree that leads to the lowest test error rate
- However, estimating the cross-validation error for every possible subtree is not feasible, since the number of possible subtrees is large

We need a way to select a small set of subtrees for consideration.

Tree Pruning

- A common strategy is to grow a very large tree T_0, and then *prune it back* in order to obtain a subtree. We use the **cost complexity pruning**, a.k.a. **weakest link pruning**.
- We consider a sequence of trees indexed by a nonnegative tuning parameter α. For each value of α find a subtree $T \subset T_0$ such that

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|,$$

  is minimized. Here $|T|$ indicates the number of leaf nodes of the tree T.
- The tuning parameter α controls a trade-off between the subtree's complexity and its fit to the training data.
- $\alpha = 0$ gives T_0.

Tree Pruning

- It turns out that as we increase α from 0, branches get pruned from the tree in a nested and predictable fashion, so obtaining the whole sequence of subtrees as a function of α is easy.

- We can select a value of α using a validation set or using cross-validation.

- We then return to the full data set and obtain the subtree corresponding to α.

Algorithm for Building a Regression Tree

Algorithm: Building a Regression Tree

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:
   a. Repeat Steps 1 and 2 on all but the kth fold of the training data.
   b. Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.

   Average the results for each value of α, and pick α to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α.

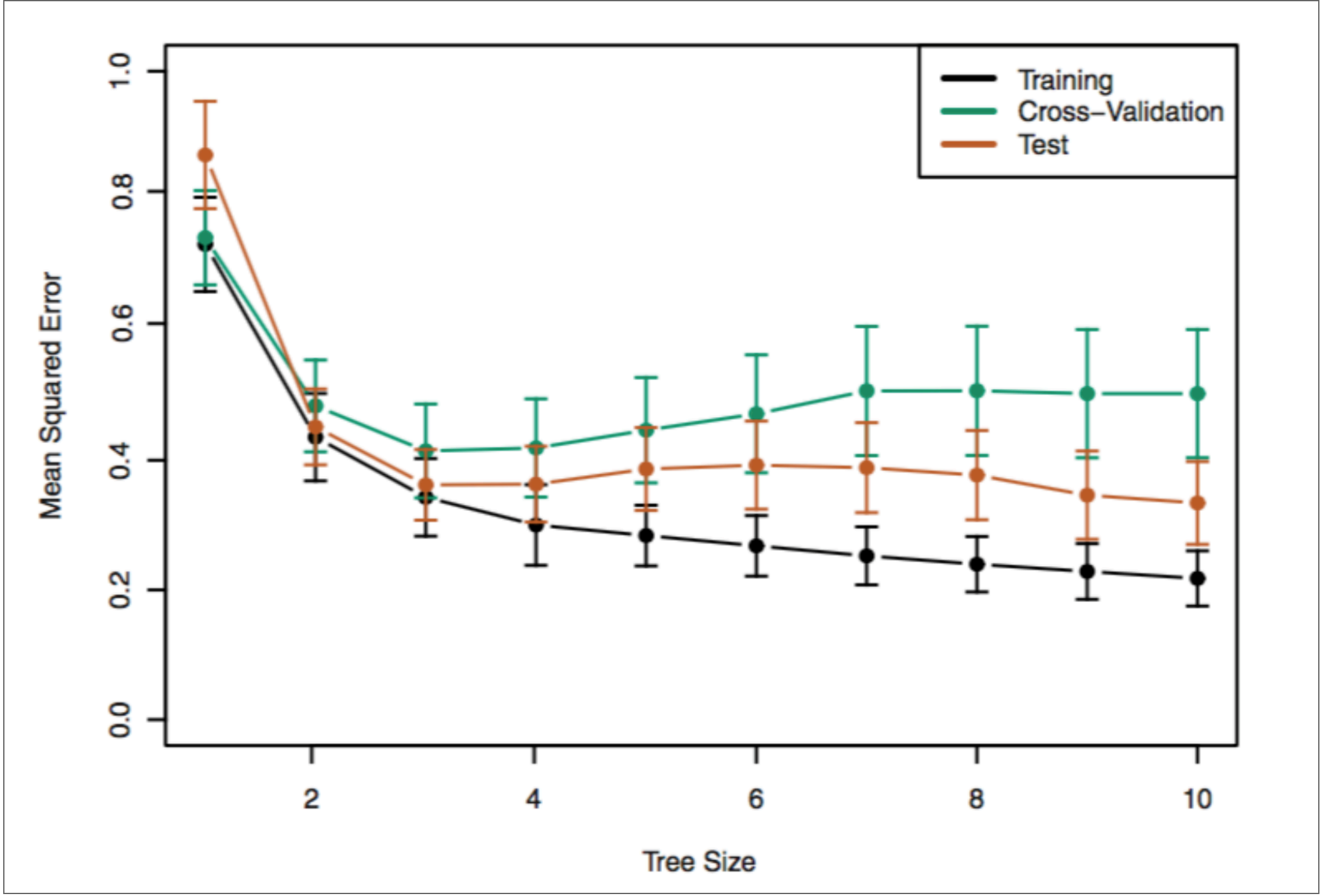
Baseball Example

- First, we randomly divided the data set in half, yielding 132 observations in the training set and 131 observations in the test set.

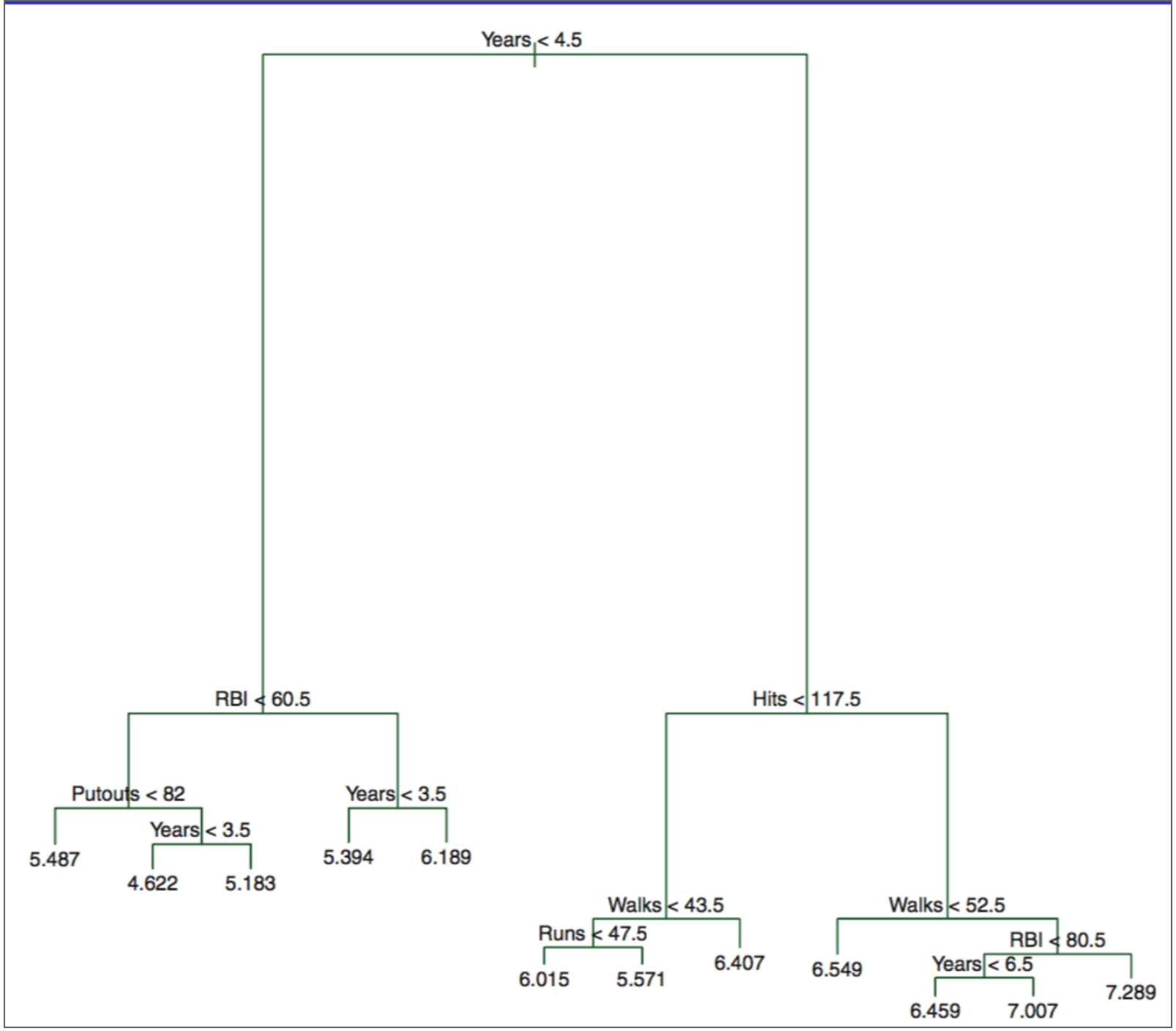
- We then built a large regression tree on the training data and varied α to create subtrees with different numbers of terminal nodes.

- Finally, we performed six-fold cross-validation in order to estimate the cross-validated MSE of the trees as a function of α.

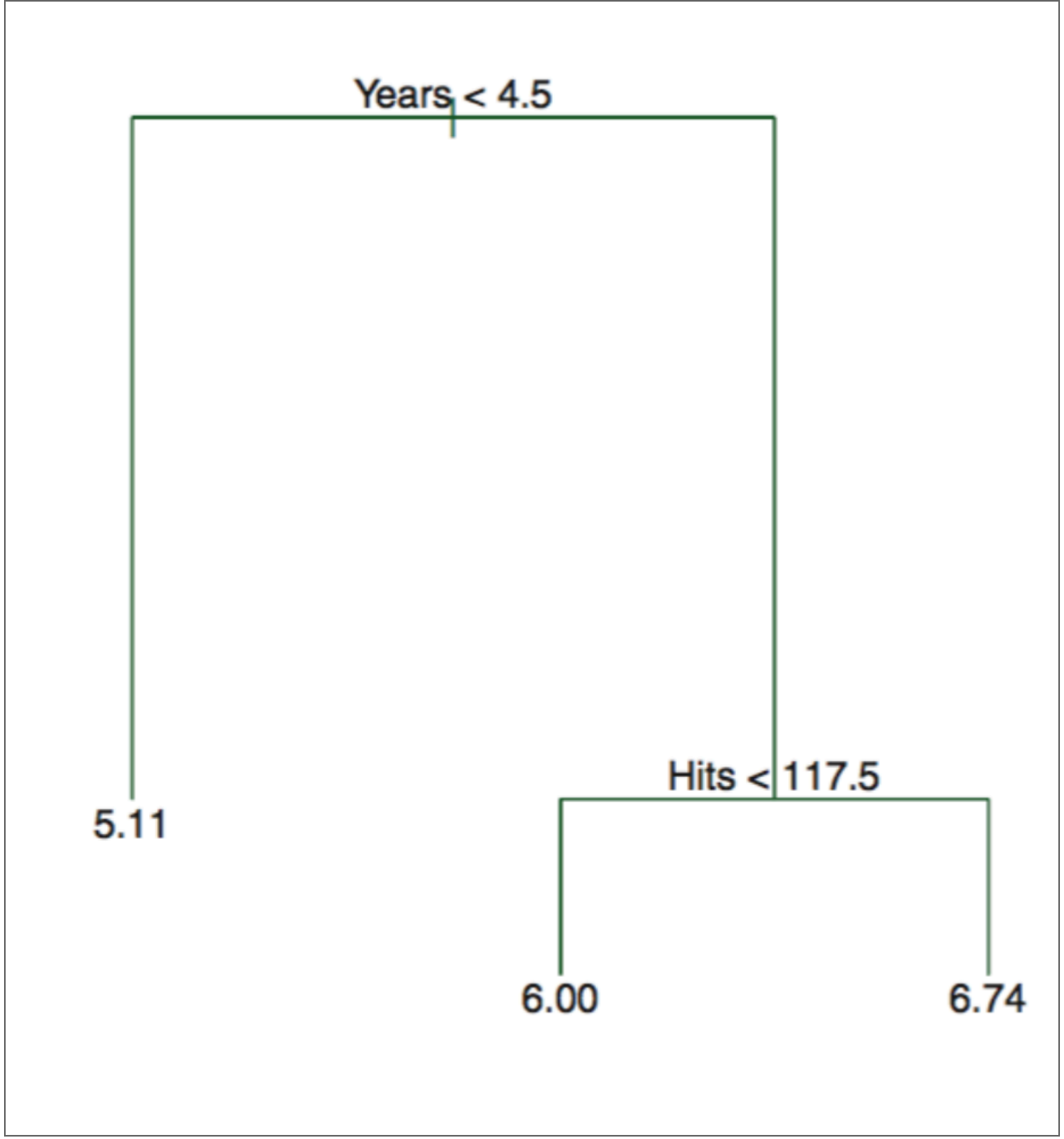
Hitters data set


Training
Cross–Validation
Test
Mean Squared Error
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
10
Tree Size

Hitters data set: the unpruned tree


Years < 4.5
RBI < 60.5
Hits < 117.5
Putouts < 82
Years < 3.5
Years < 3.5
Walks < 43.5
Walks < 52.5
Runs < 47.5
RBI < 80.5
Years < 6.5
5.487
4.622
5.183
5.394
6.189
6.015
5.571
6.407
6.549
6.459
7.007
7.289

Hitters data set: the pruned tree


Years < 4.5
5.11
Hits < 117.5
6.00
6.74

Classification Trees

- A classification tree is very similar to a regression tree, except that it is used to predict a qualitative response rather than a quantitative one.

- For a classification tree, we predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.

- Just as in the regression setting, we use recursive binary splitting to grow a classification tree.

- But, RSS cannot be used as a criterion for making the binary splits. **What metric should we use instead?**

Metrics used for classification trees

- A natural alternative to RSS is the **0-1 misclassification error rate**. For region R_m with K classes of labels: for $k \in \{1, \ldots, K\}$,

$$\hat{p}_{mk} = \frac{1}{|R_m|} \sum_{i: x_i \in R_m} \mathbf{1}\{y_i = k\}$$

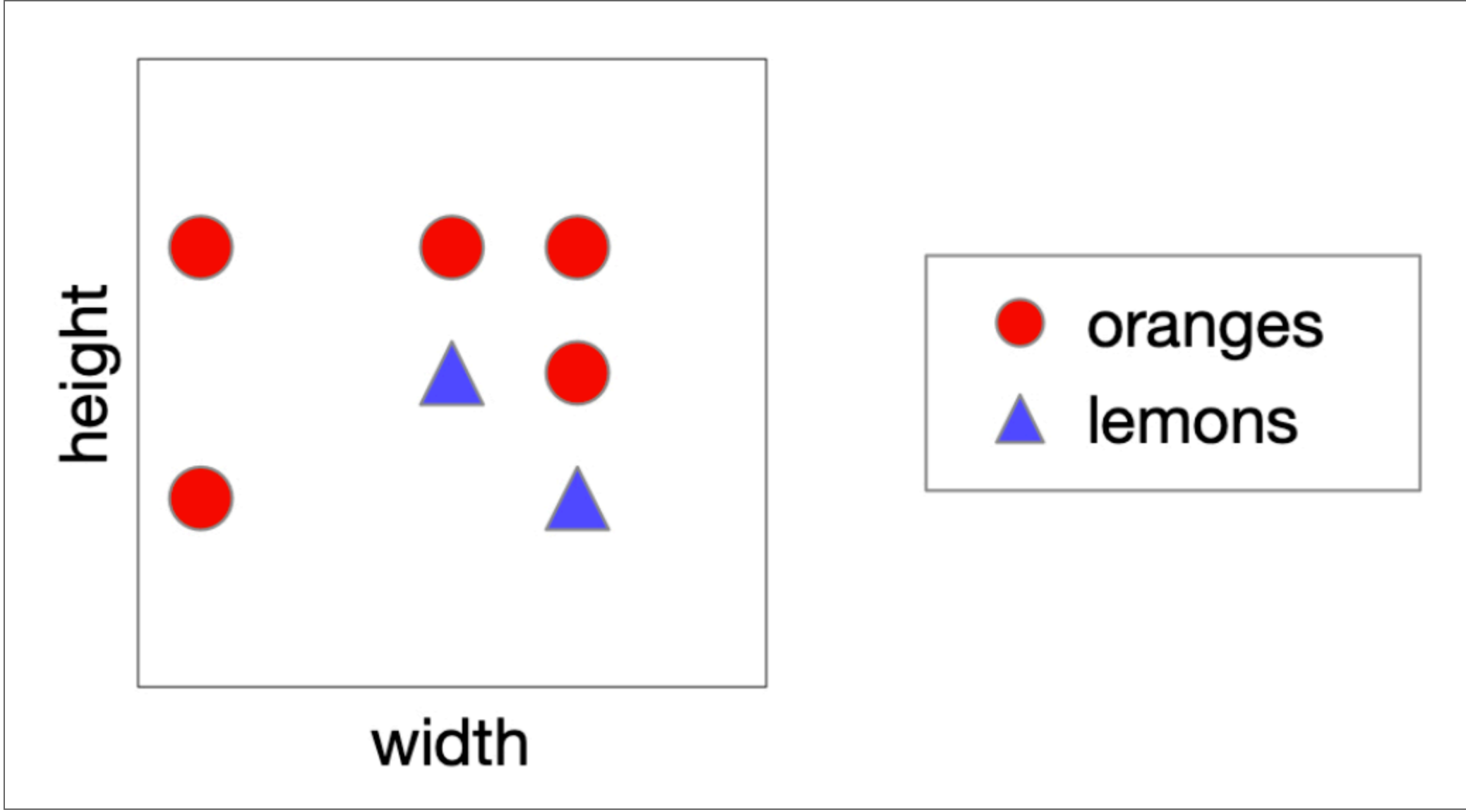
- The misclassification error rate in region R_m is

$$1 - \max_{k \in \{1, \ldots, K\}} \hat{p}_{mk}.$$

- However misclassification error is not sufficiently sensitive for the change of $\hat{p}_{mk}$, hence not suitable for finding the splits for **building** classification trees.

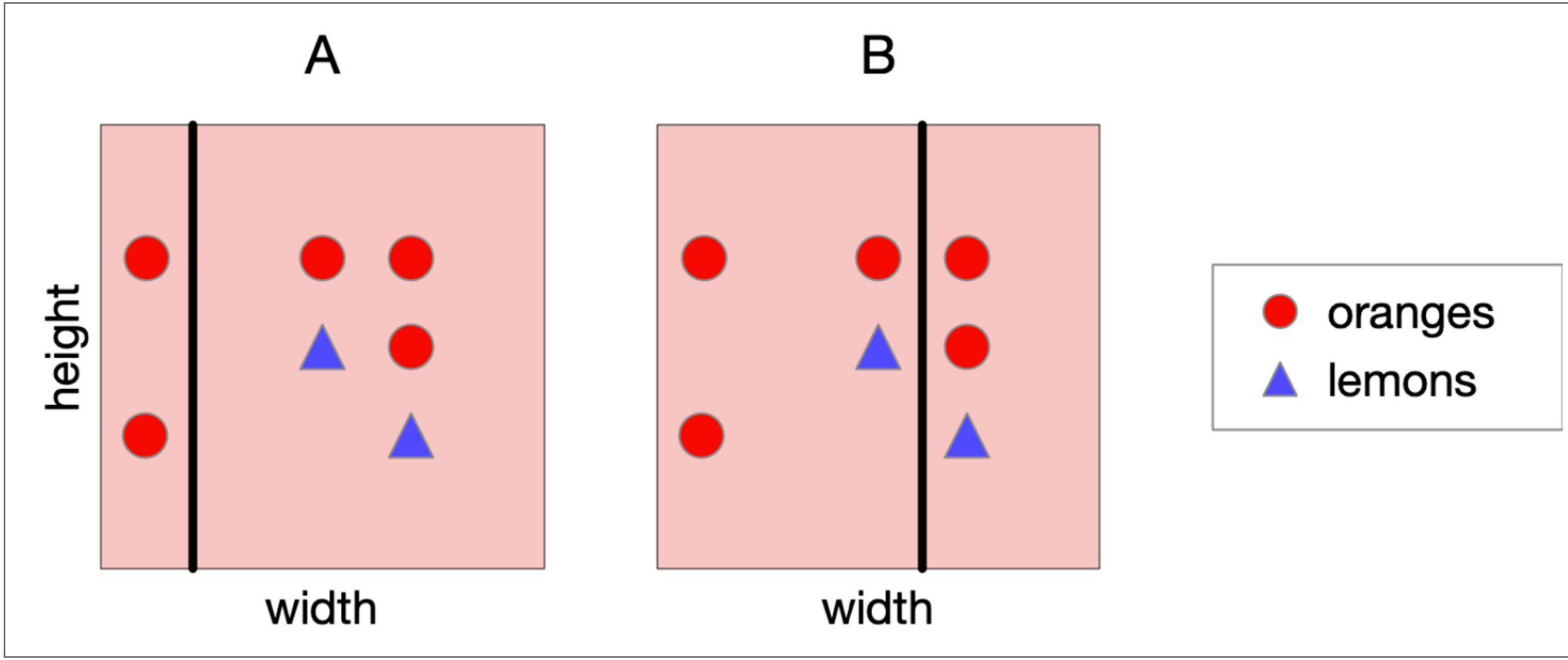
Choosing a Good Split

Consider the following data. Let’s split on width.


height
width
oranges
lemons

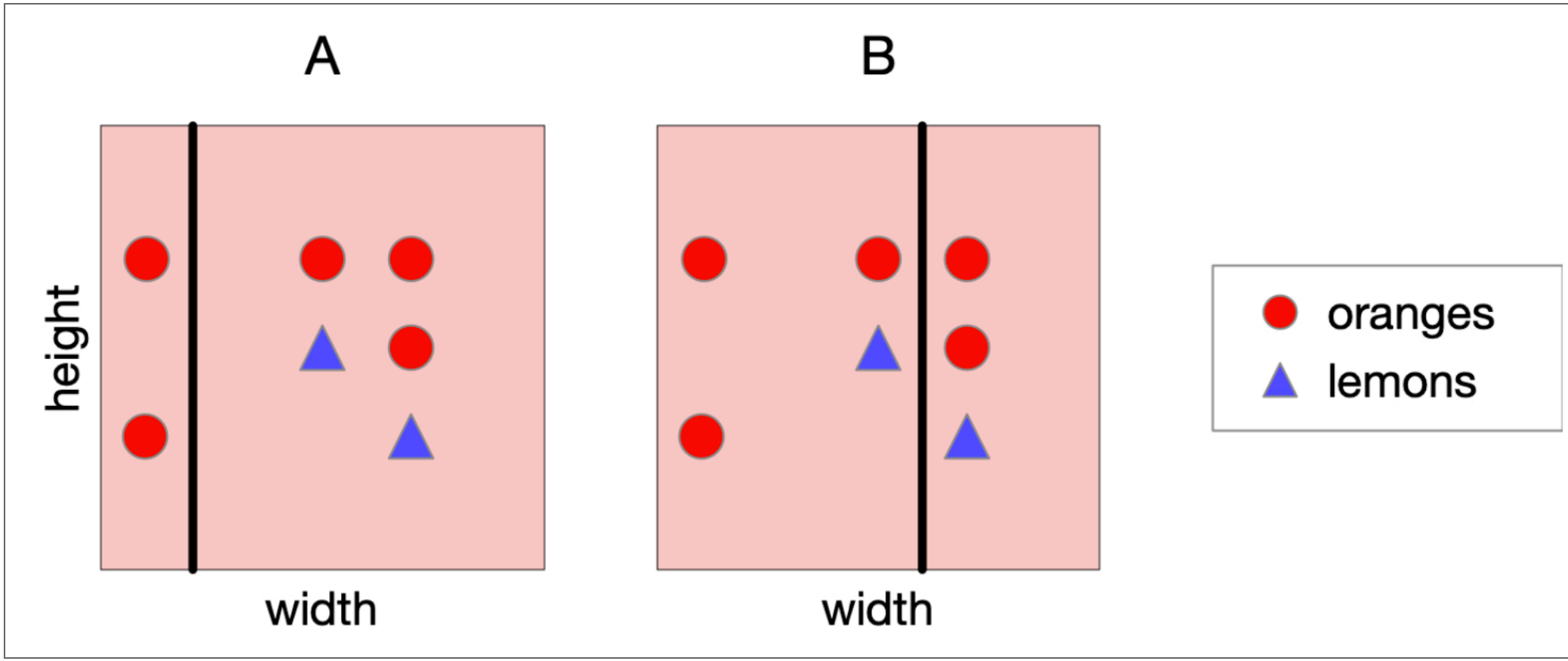
Choosing a Good Split

Recall: classify by majority.



A and B have the same misclassification rate, so which is the best split? Vote!

Choosing a Good Split



Case A feels like a better split, because the left-hand region is very certain about whether the fruit is an orange.

Can we quantify this?

Quantifying Uncertainty

- The **entropy** of a discrete random variable is a number that quantifies the **uncertainty** inherent in its possible outcomes.

- The mathematical definition of entropy that we give in a few slides may seem arbitrary, but it can be motivated axiomatically.
- If you're interested, check: Information Theory by Robert Ash.

- To explain entropy, consider flipping two different coins.

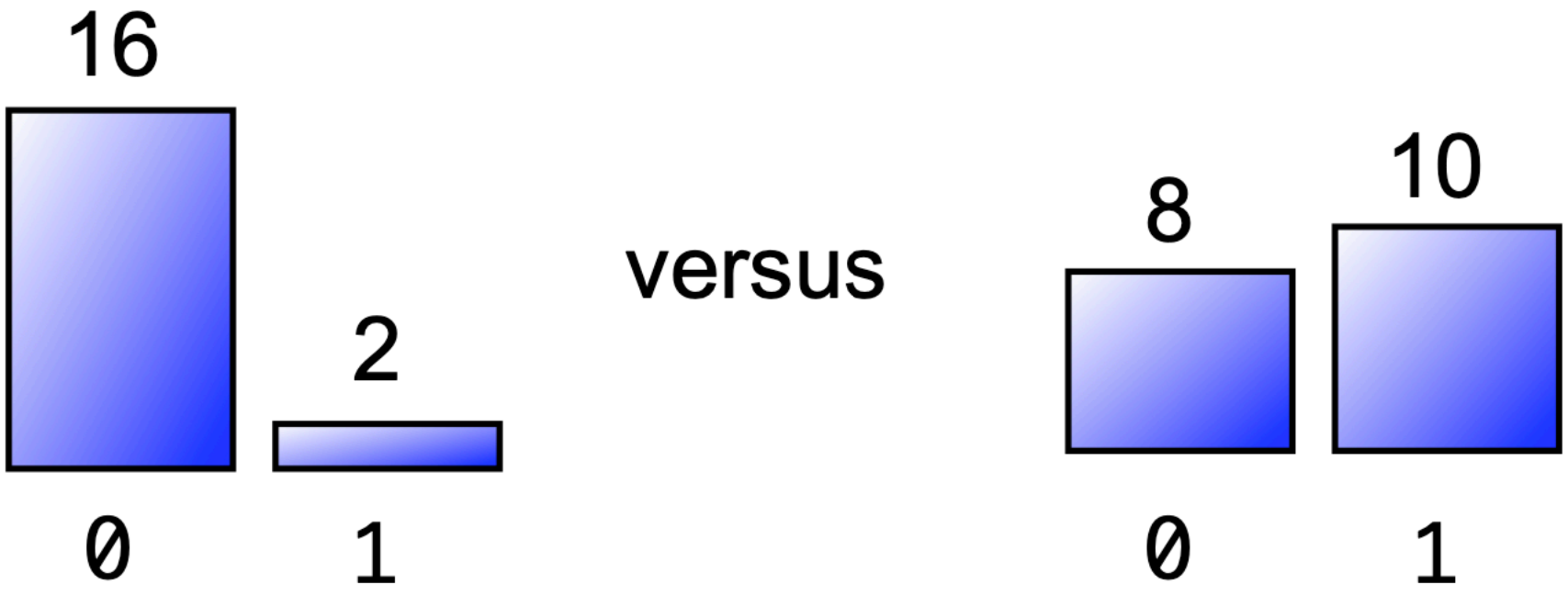
We Flip Two Different Coins

Sequence 1:

0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 ... ?

Sequence 2:

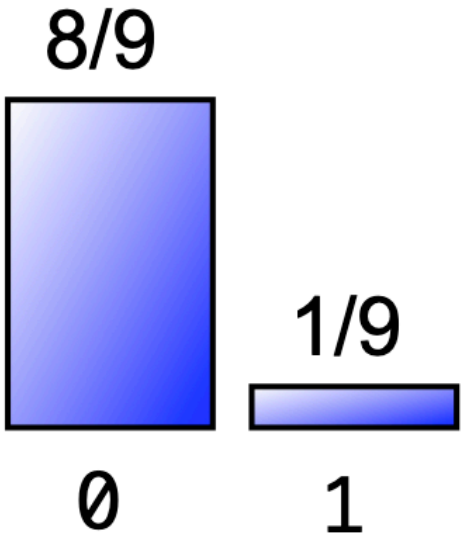
0 1 0 1 0 1 1 1 0 1 0 0 1 1 0 1 0 1 ... ?



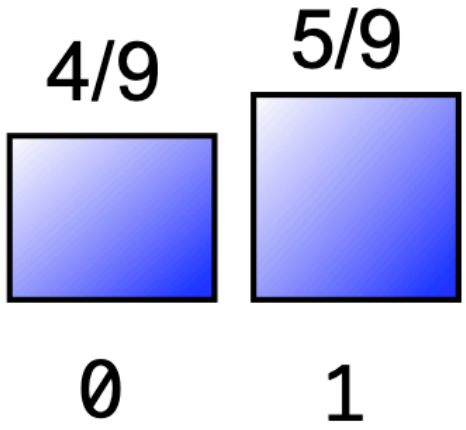
Quantifying Uncertainty

- The entropy of a loaded coin with probability p of heads is given by

$$-p\log_2(p) - (1-p)\log_2(1-p)$$



$$-\frac{8}{9}\log_2\frac{8}{9} - \frac{1}{9}\log_2\frac{1}{9} \approx \frac{1}{2}$$



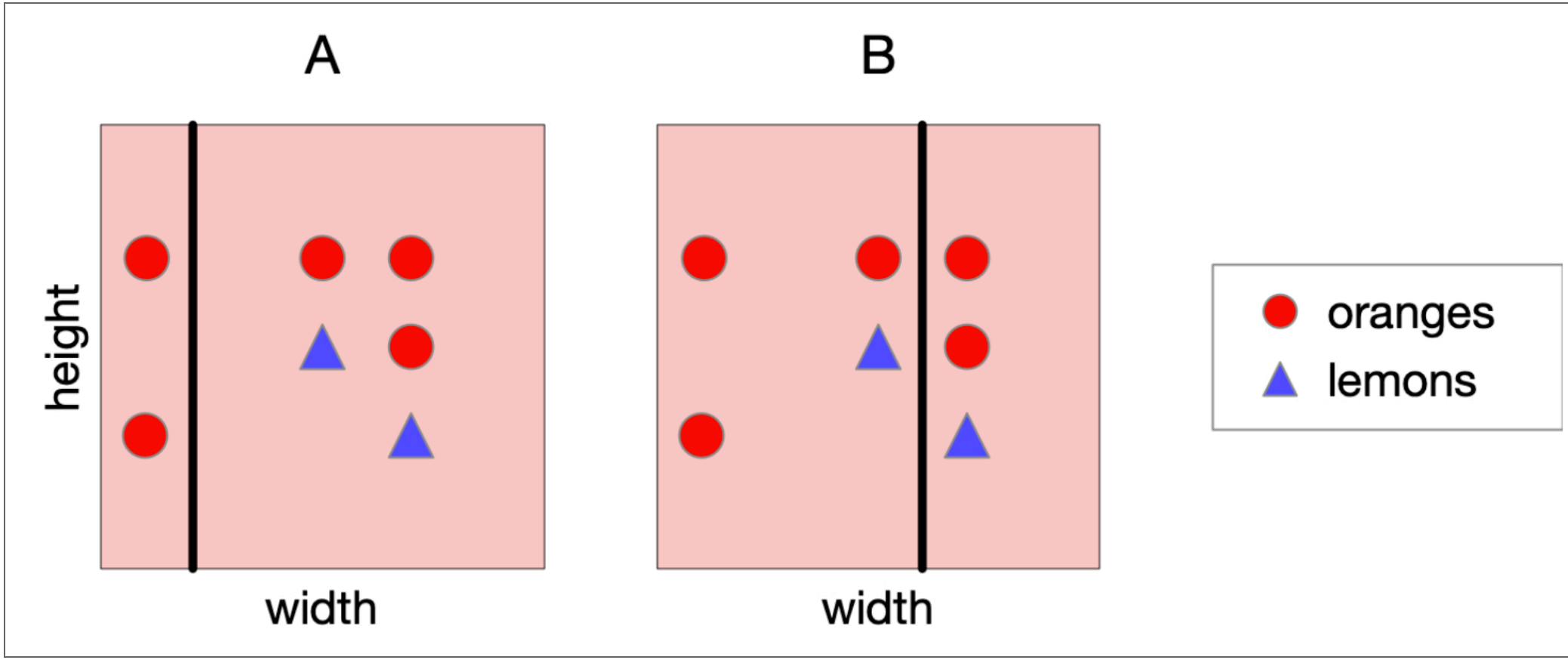
$$-\frac{4}{9}\log_2\frac{4}{9} - \frac{5}{9}\log_2\frac{5}{9} \approx 0.99$$

- Notice: the coin whose outcomes are more certain has a lower entropy.

- In the extreme case $p = 0$ or $p = 1$, we were certain of the outcome before observing. So, we gained no certainty by observing it, i.e., entropy is 0.

Revisiting Our Original Example

Let's compute the entropies associated with Split B.

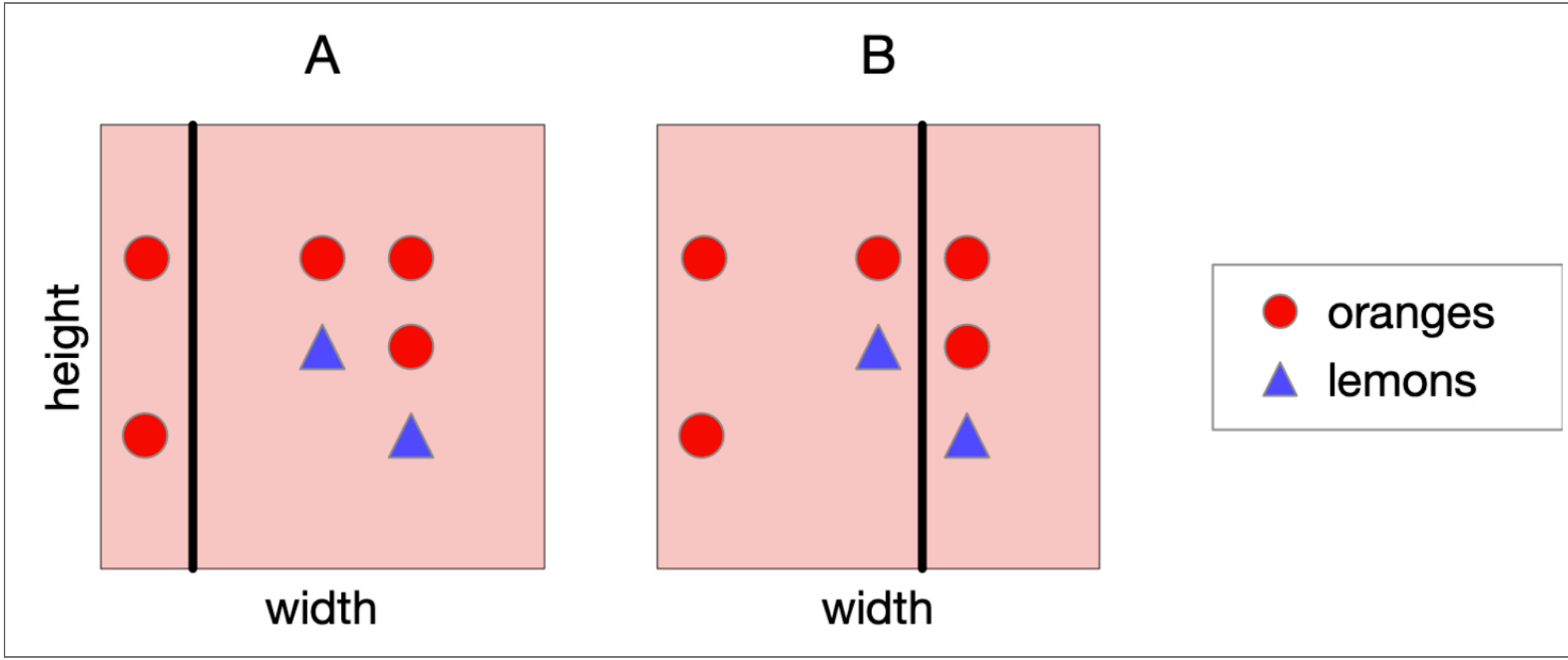


- Left branching: $\hat{p}_{\text{orange}} = 3/4$ and $\hat{p}_{\text{lemon}} = 1/4$. Its entropy is $-\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} \approx 0.81$.
- Right branching: $\hat{p}_{\text{orange}} = 2/3$ and $\hat{p}_{\text{lemon}} = 1/3$. Its entropy is $-\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \approx 0.92$

- The summation of entropies is 1.73.

Revisiting Our Original Example

What is the information gain of split A? Very informative!



- Left branching: $\hat{p}_{\text{orange}} = 1$ and $\hat{p}_{\text{lemon}} = 0$. Its entropy is 0.
- Right branching: $\hat{p}_{\text{orange}} = 3/5$ and $\hat{p}_{\text{lemon}} = 2/5$. Its entropy is $-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} \approx 0.97$

- The summation of entropies is 0.97. So we have less uncertainty for the second split. Uncertainty can also be understood as **node purity**.

Deviance and Gini index

- For a region R_m, an entropy based metric is called **Deviance**, and is given by

$$-\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk},$$

- Another popular metric is **Gini index**, defined by

$$\sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}),$$

  a measure of total variance across the K classes.

- Both Deviance and Gini index are small if all of the $\hat{p}_{mk}$'s are close to zero or one. They are measures of **node purity** – a small value indicates that a node contains observations that most of them come from a single, dominant class.

Misclassification error rate, Gini index and Deviance

- It turns out that Gini index and Deviance are quite similar numerically.

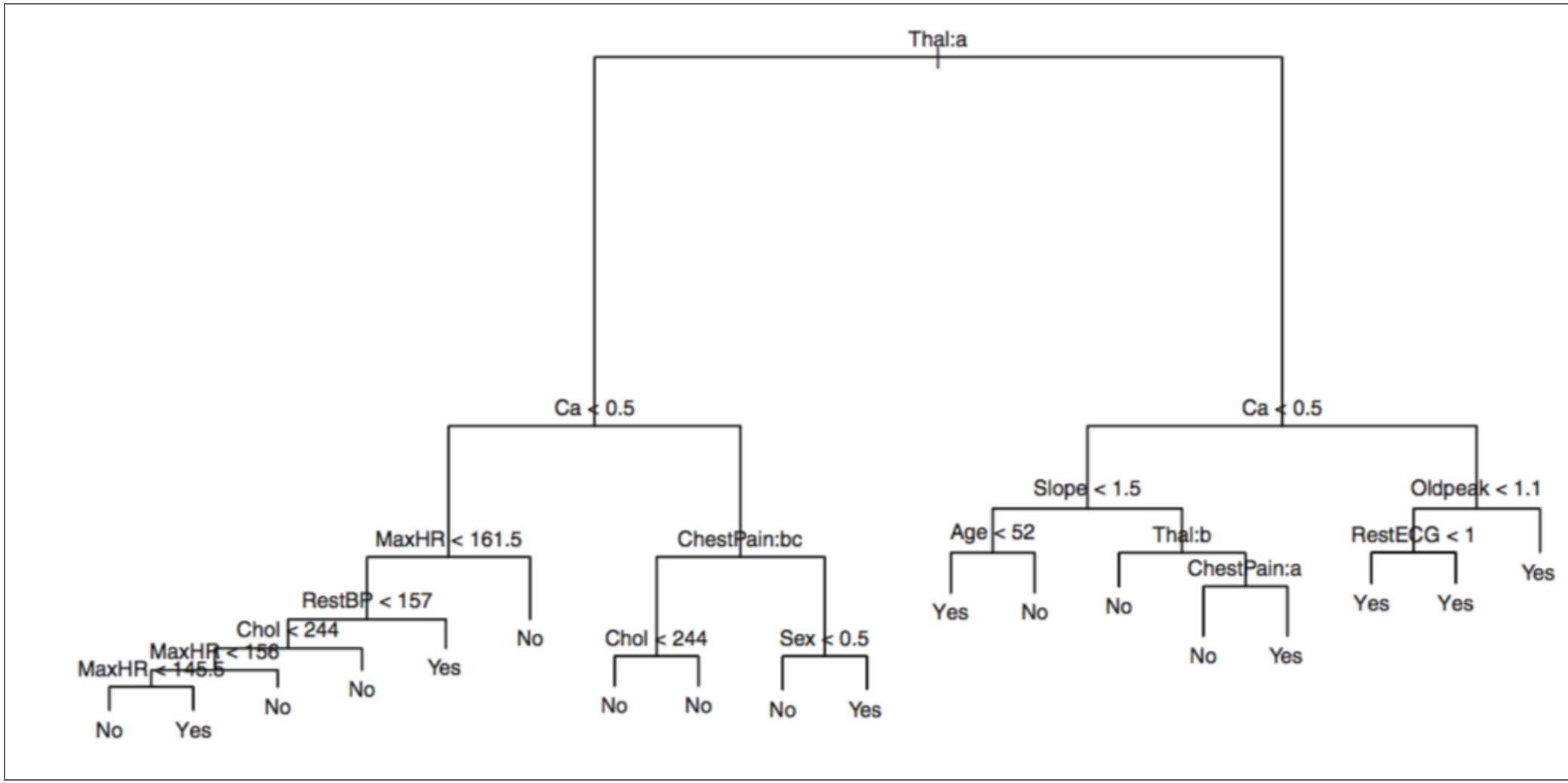
- On the one hand, for **building** a classification tree, either Gini index or Deviance can be used for finding best splits, since they are more sensitive to node purity than the misclassification error rate.

- On the other hand, all Gini index, Deviance and misclassification error can be used for **pruning** the tree, but the misclassification error is preferable if classification accuracy is the final goal.

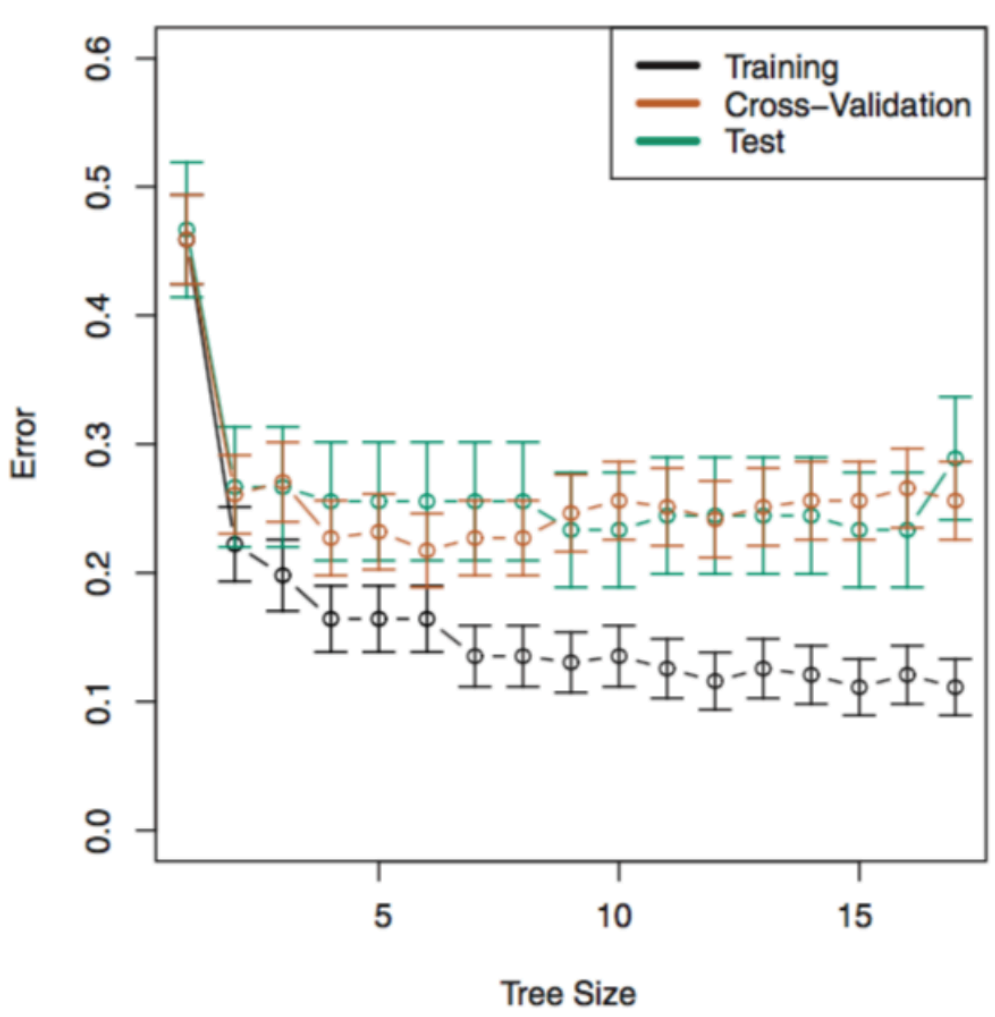
Heart Data

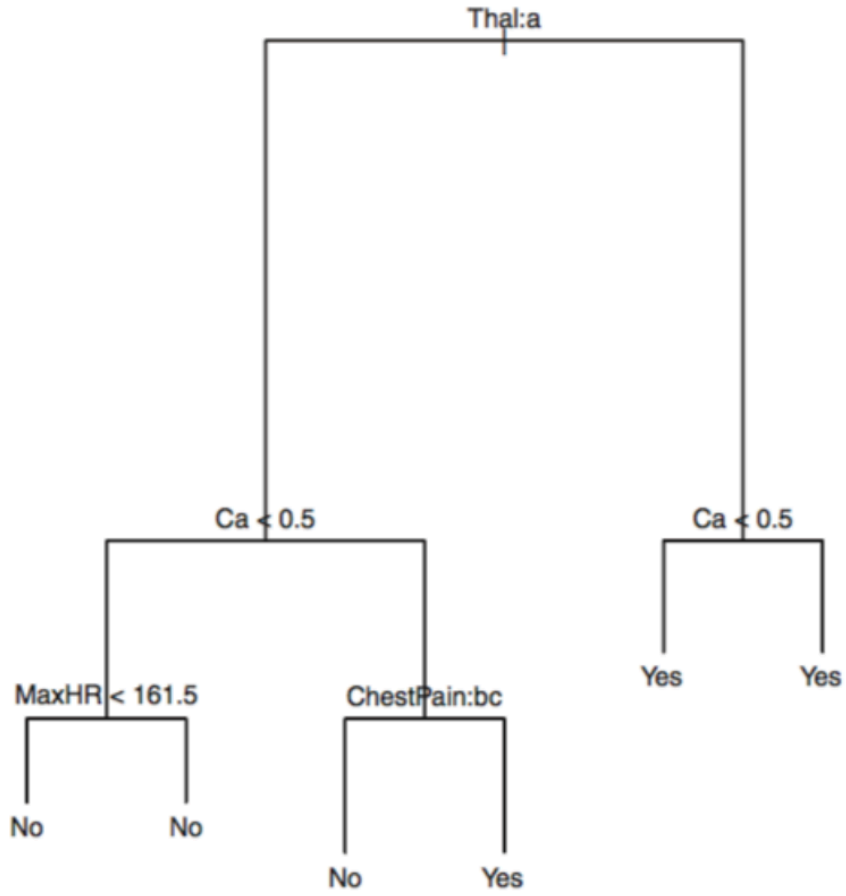
- These data contain a binary outcome HD for 303 patients who presented with chest pain.
- An outcome value of Yes indicates the presence of heart disease based on an angiographic test, while No means no heart disease.
- There are 13 features including Age, Sex, Chol (a cholesterol measurement), Thal (maximum heart rate achieved) and other heart and lung function measurements.
- Cross-validation yields a tree with six terminal nodes.

Heart Data: the unpruned fitted decision tree.



Heart Data





Left: Cross-validation error, training, and test error, for different sizes of the pruned tree.

Right: The pruned tree corresponding to the minimal CV error.

Trees Versus Linear Models

- The linear model says

$$f(X) = \beta_0 + \sum_{j=1}^{p} \beta_j X_j,$$

whereas the regression tree says

$$f(X) = \sum_{m=1}^{J} c_m \mathbf{1}\{X \in R_m\}.$$

- If $f(X) \approx X^\top \beta$, then linear regression will likely work well.
- If instead there is a highly non-linear and complex relationship between X and Y, then decision trees performs better.
- The relative performances of tree-based and classical approaches can be assessed by estimating the test error, using either cross-validation or the validation set approach.

Advantages and Disadvantages of Trees

- Trees are self-explanatory. In fact, they are even easier to explain than linear regression!
- Some people believe that decision trees more closely mimic human decision-making.
- Trees can easily handle qualitative features without the need to create dummy variables.

- But, trees generally do not have the same predictive power as other regression and classification approaches.
- Relatedly, trees could be very non-robust. In other words, a small change in the data can lead to a drastically different estimated tree.

- Nevertheless, by aggregating many trees (via bagging, random forests, and boosting), the predictive performance can be substantially improved.

Bootstrap

Bootstrap

- Bootstrap is a widely used **resampling** (of the available data) approach!

- It can be used to assess the uncertainty of basically **any** statistical procedure.
- For instance, it can be used to estimate the standard errors of the estimated coefficients of a linear model.
- Much more generally, it can even estimate the whole distribution of the estimated coefficients of a linear model.

- Its validity is backed up by a very general theory!

An example

- Suppose that we wish to invest 10K dollars in two financial assets that yield returns of X and Y, respectively, where X and Y are random quantities.

- For any $\alpha \in [0, 1]$, we will invest a fraction α of our money in X, and will invest the remaining $(1 - \alpha)$ in Y.

$$[\alpha X + (1 - \alpha)Y] \times 10{,}000$$

- We wish to choose α to minimize the total risk, or variance, of our investment, that is,

$$\min_{\alpha \in [0,1]} \mathrm{Var}(\alpha X + (1 - \alpha)Y).$$

An example

- One can show that the value of α that minimizes the risk is given by

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}},$$

  where $\sigma_Y^2 = \mathrm{Var}(Y)$, $\sigma_X^2 = \mathrm{Var}(X)$ and $\sigma_{XY} = \mathrm{Cov}(X, Y)$.
- If we have past observations $(x_1, y_1), \ldots, (x_{100}, y_{100})$, we can estimate α by

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}.$$

- How to estimate the variance of the estimator $\hat{\alpha}$?

An Oracle Approach

If we know the distribution of X and Y (usually unknown in reality), we can estimate the variance of the estimator $\hat{\alpha}$ by the following strategy.

- We simulate 100 paired observations of X and Y and compute

$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}.$$

- We repeat this procedure 1000 times, and get $\hat{\alpha}_1, \ldots, \hat{\alpha}_{1000}$.
- We estimate $\mathrm{Var}(\hat{\alpha})$ by

$$\frac{1}{1000-1}\sum_{r=1}^{1000}(\hat{\alpha}_r - \bar{\alpha})^2, \qquad \text{where} \quad \bar{\alpha} = \frac{1}{1000}\sum_{r=1}^{1000}\hat{\alpha}_r.$$

Question: Is this feasible in practice?

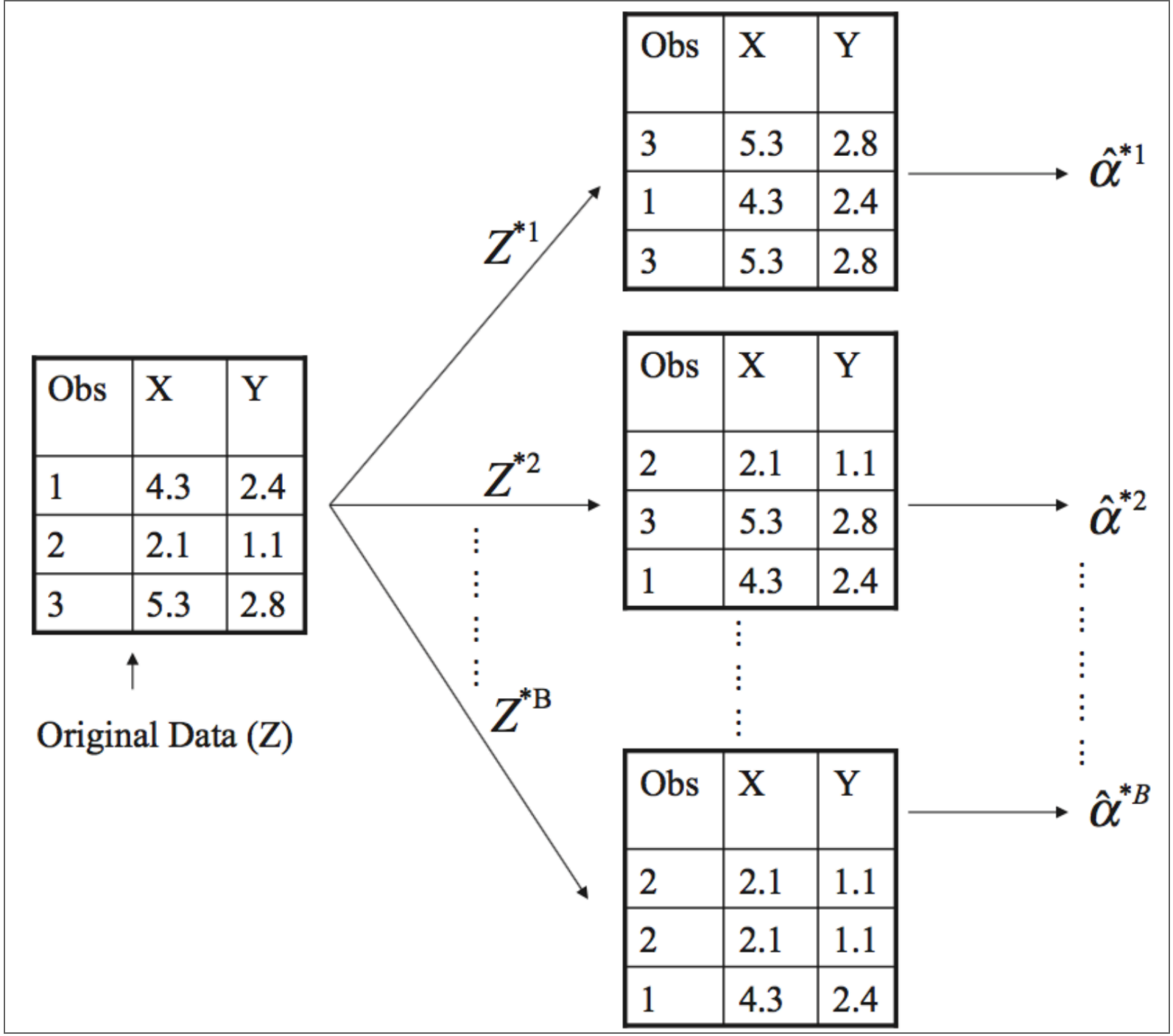
Bootstrap

- The bootstrap approach re-samples from the **original data set** to mimic the process of obtaining new data sets in order to quantifty the uncertainty of a given procedure.

- Specifically, for a specified B (for instance, B = 1000) number of repetitions, we repeatedly sample **the same amount of observations** from the original data set **with replacement**.

- As a result, data set from bootstrap might contain some observations more than once, or zero time.

Simple illustration of Bootstrap


Obs X Y
1 4.3 2.4
2 2.1 1.1
3 5.3 2.8
Original Data (Z)
Z^{*1}
Z^{*2}
Z^{*B}
Obs X Y
3 5.3 2.8
1 4.3 2.4
3 5.3 2.8
Obs X Y
2 2.1 1.1
3 5.3 2.8
1 4.3 2.4
Obs X Y
2 2.1 1.1
2 2.1 1.1
1 4.3 2.4
$\hat{\alpha}^{*1}$
$\hat{\alpha}^{*2}$
$\hat{\alpha}^{*B}$

Bootstrap

Now let us apply bootstrap to estimate $\mathrm{Var}(\hat{\alpha})$:

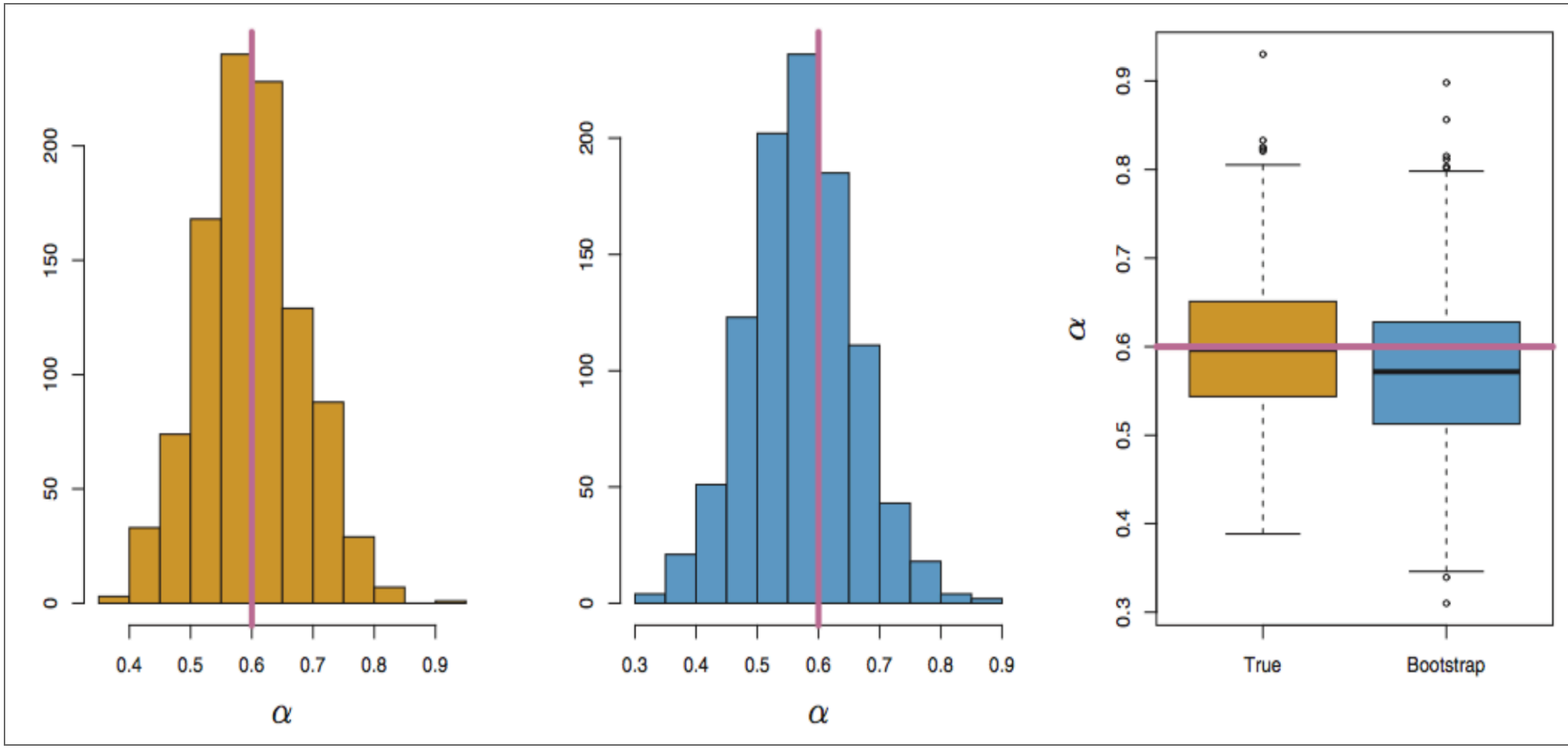
- We denote the first bootstrap data set by Z^{*1}, and use Z^{*1} to construct the estimate of α, denoted by $\hat{\alpha}^{*1}$.

- This procedure is repeated B (say, $B = 1000$): specifically we simulate B different bootstrap data sets, $Z^{*1}, \ldots, Z^{*B}$ and B corresponding estimates $\hat{\alpha}^{*1}, \ldots, \hat{\alpha}^{*B}$.

- We estimate $\mathrm{Var}(\hat{\alpha})$ by the sample variance of $\hat{\alpha}^{*1}, \ldots, \hat{\alpha}^{*B}$:

$$\frac{1}{B-1}\sum_{b=1}^{B}(\hat{\alpha}^{*b} - \bar{\alpha}^{*})^2, \qquad \text{where} \quad \bar{\alpha}^{*} = \frac{1}{B}\sum_{b=1}^{B}\hat{\alpha}^{*b}.$$

Example



Left: The histogram of estimates of α obtained by generating 1,000 simulated data sets from the true population.

Center: The histogram of estimates of α obtained from 1,000 bootstrap samples from a single data set.

Right: Boxplots for estimates of α displayed in the left and center panels.

Bootstrap for quantifying the uncertainty of the OLS estimator

Given the data $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, the OLS gives

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

Statistical property of $\hat{\beta} \in \mathbb{R}^p$ consists of

- its mean
- its covariance
- Recall that analyses of $\hat{\beta}$ such as its mean and covariance are only available under linear model assumption.
- its higher moments
- its whole distribution

Bootstrap can be used to estimate **all above**!

Bootstrap for quantifying the uncertainty of the OLS estimator

Given the data $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, the OLS gives

$$\hat{\beta} = (X^\top X)^{-1} X^\top y.$$

For $b = 1, \ldots, B$,

1. obtain the bootstrap sample $D^b = (X^b, y^b)$
2. compute $\hat{\beta}^b = (X^{b\top} X^b)^{-1} X^{b\top} y^b$.

Now for $\hat{\beta}_j$, the bootstrap estimates $\hat{\beta}_j^1, \ldots, \hat{\beta}_j^B$ serve as "samples" of $\hat{\beta}_j$.

Bootstrap for quantifying the uncertainty of the OLS estimator

For instance,

- the mean of $\hat{\beta}_j$ can be estimated by $\frac{1}{B}\sum_{b=1}^{B}\hat{\beta}_j^b$.
- the variance of $\hat{\beta}_j$ can be estimated by $\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\beta}_j^b - \frac{1}{B}\sum_{b=1}^{B}\hat{\beta}_j^b\right)^2$.

- You can also estimate quantiles of the distribution of $\hat{\beta}_j$.
- In fact, you can estimate the whole distribution of $\hat{\beta}_j$.

Summary

- **Decision Trees:** Partition the feature space into regions, making predictions using the mean response (regression) or majority class (classification).
- **Building Trees:** Recursive binary splitting uses RSS for regression and measures of node purity (Gini index or deviance) for classification.
- **Pruning:** Large trees can overfit. Cost-complexity pruning uses a tuning parameter α, typically selected using cross-validation, to balance complexity and predictive performance.
- **Strengths and Limitations:** Trees are interpretable and capture non-linear relationships, but can be unstable and have lower predictive accuracy than other methods.
- **Bootstrap:** Resampling observations with replacement allows us to approximate the sampling distribution of an estimator and quantify its uncertainty (e.g., standard errors and variance).

Announcements: Next week is reading week!
After Reading Week: More tree based methods and midterm

Speaker notes